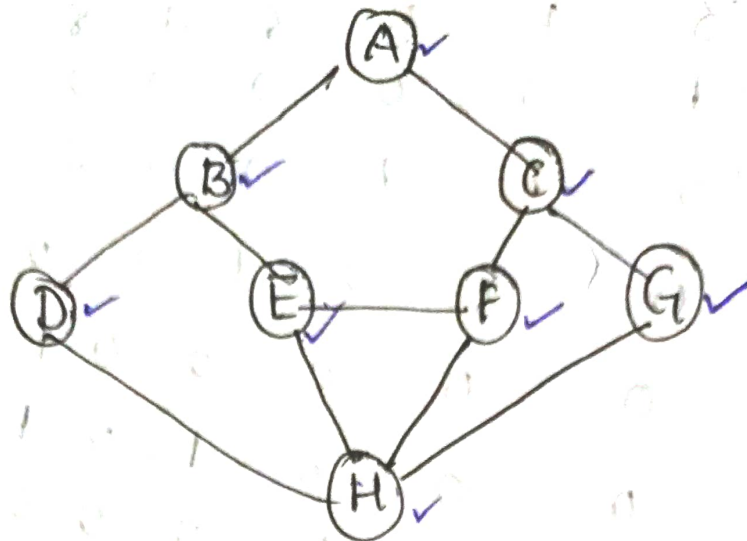


6



### Adjacency list:

A: [B, C]

B: [A, D, E]

C: [A, F, G]

D: [B, H]

E: [B, F, H]

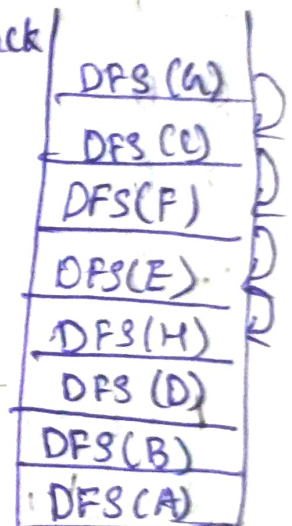
F: [C, E, H]

G: [C, H]

H: [D, E, F, G]

Steps: DFS order: A, B, D, H, E, F, C, G

1. All vertices are unvisited and the callstack is empty. DFS starts from vertex A.
2. call DFS(A)
3. DFS(A) is pushed onto the call stack
4. Visit A: mark it is visited and add it to the DFS order.
5. check neighbour B of A : not visited, so call DFS(B).
6. DFS(B) is pushed on to callstack.
7. Visit B: mark it is visited and add it to the DFS order.
8. check neighbour A of B : already visited, so skip it.
9. check neighbour D of B : not visited, so call DFS(D).
10. DFS(D) is pushed on to the callstack.
11. Visit D: mark it is visited and add it to the DFS order.
12. check neighbour B of D : already visited, so skip it.
13. check neighbour H of D : not visited, so call DFS(H).
14. Visit H: Mark it is visited and add it to the DFS order.
15. check neighbour D of H : already visited, so skip it.
16. check neighbour E of H : not visited, so call DFS(E)
17. Visit E: Mark it is visited and add it to the DFS order.



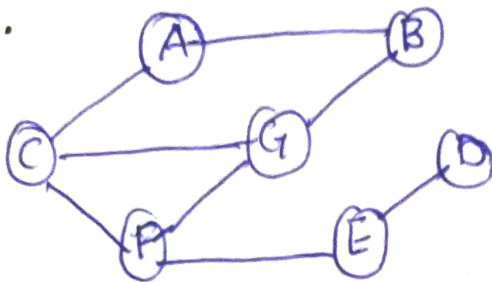
Call stack

18. Check neighbour B of E: already visited, so skip it.
19. Check neighbour F of E: not visited, so call DFS(F)
20. Visit F: Mark it is visited and add it to the DFS order.
21. Check neighbour C of F: not visited, so call DFS(C)
22. Visit C: Mark it is visited and add it to the DFS order.
23. Check neighbour A of C: already visited, so skip it.
24. Check neighbour F of C: already visited, so skip it
25. Check neighbour G of C: not visited, so call DFS(G)
26. Visit G: Mark it is visited and add it to the DFS order
27. Check neighbours C & H of G: already visited, skip it. <sup>pop(G)</sup>
28. Back to DFS(C): all neighbours are visited, skip it. <sup>pop(C)</sup>
29. Back to DFS(F): all neighbours are visited, skip it. <sup>pop(F)</sup>
30. Back to DFS(E): all neighbours are visited, skip it. <sup>pop(E)</sup>
31. Back to DFS(H): all neighbours are visited, skip it. <sup>pop(H)</sup>
32. Back to DFS(D): all neighbours are visited, skip it. <sup>pop(D)</sup>
33. Back to DFS(B): all neighbours are visited, skip it. <sup>pop(B)</sup>
34. Back to DFS(A): all neighbours are visited, skip it and DFS(A) returns and is popped.

BFS :-

→ BFS visits a graph level by level. first start vertex, then all of its neighbours, then their neighbours and so on. It uses a queue and a visited set.

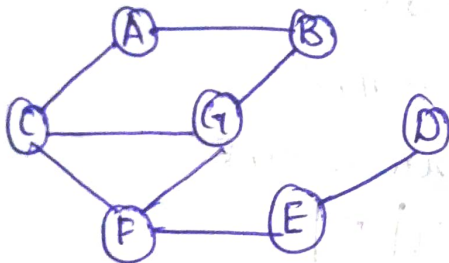
1.



Adjacency list

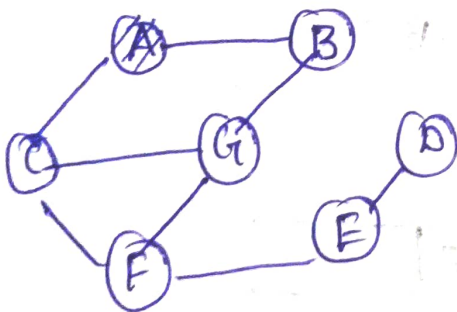
A [B, C]  
B [A, G]  
C [A, G, F]  
D [E]  
E [D, F]  
G [B, C, F]  
F [G, C, E]

Step 1 :-



Queue  
[A]  
Enqueue A

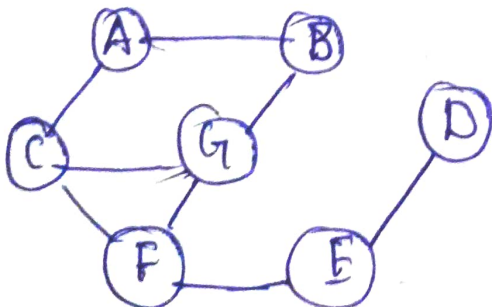
Step 2 :- Dequeue ~~front~~ <sup>front</sup>



front  
A [ ] [ ] [ ] [ ]  
O/P - A

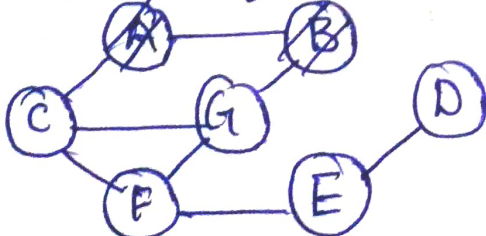
Step 3 :- Unvisited neighbours of A → B, C

Enqueue B, C



[B] [C] [ ] [ ]  
Front O/P: A

Step 4 :- Dequeue



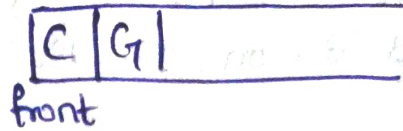
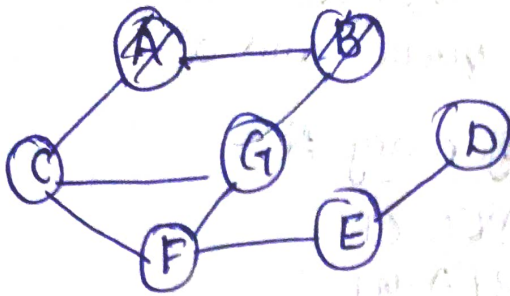
[B] [ ] [ ] [ ]  
Front

O/P :- A, B

Step 5: Unvisited Neighbours of

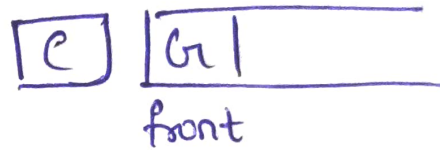
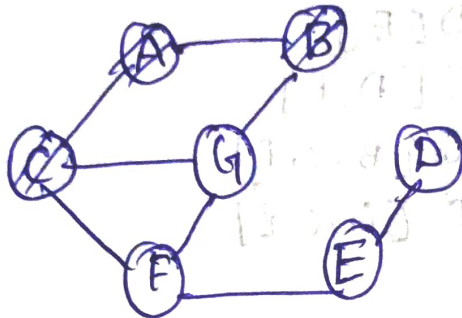
$B \rightarrow G$

Enqueue  $G$



O/P: A B

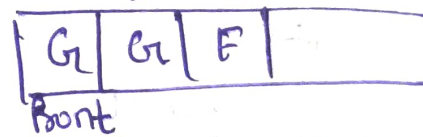
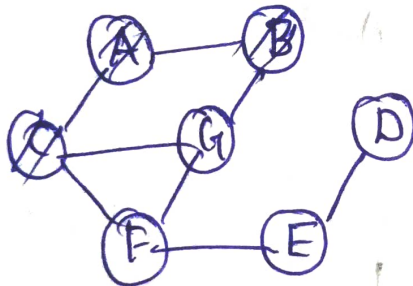
Step 6: Dequeue front



O/P: A B C

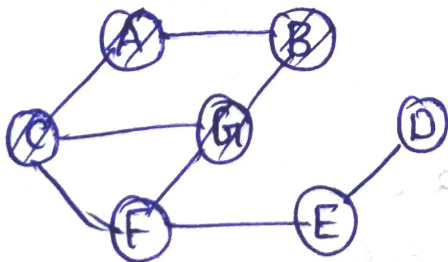
Step 7: Unvisited Neighbours of  $C \rightarrow G, F$

Enqueue  $G, F$



O/P: A B C

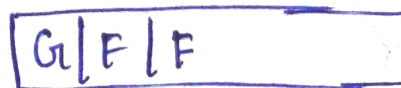
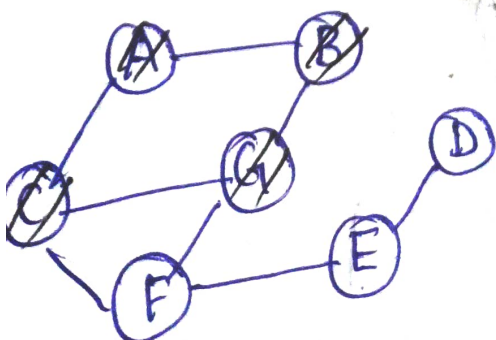
Step 8: Dequeue front



O/P: A B C G

Step 9: Unvisited Neighbours of  $G \rightarrow F$

enqueue  $F$

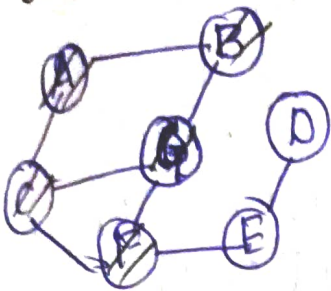


O/P: A B C G

Step 10: Dequeue front

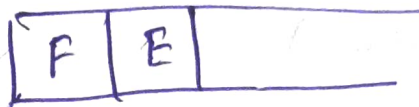


Step 11: Dequeue front



O/P: A B C G F

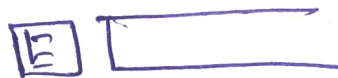
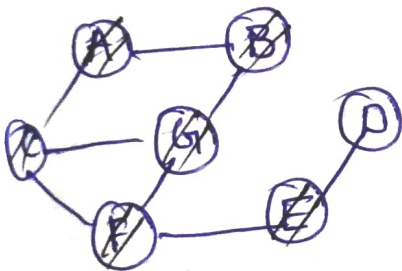
Step 12: Dequeue front  
Enqueue E



Step 13: Dequeue front



Step 14: Dequeue Front



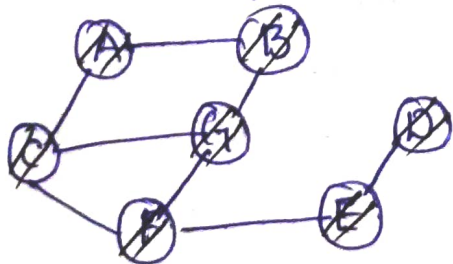
O/P: A B C G F E

Step 15: Unvisited neighbours of C → D  
Enque D



front.

Step 16: Dequeue front

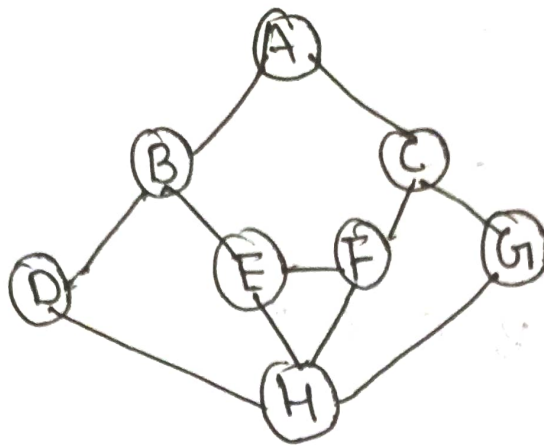


O/P:

A B C G F E D

BFS (2):

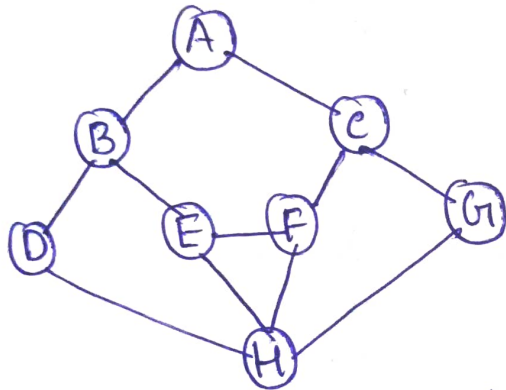
1)



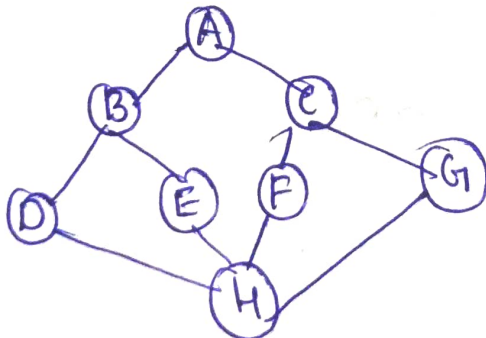
Adjacency List

A [B, C]  
 B [D, E, A]  
 C [A, F, G]  
 D [B, H]  
 E [B, F, H]  
 F [C, E, H]  
 G [C, H]  
 H [E, F, D, G]

Step 1:- Enqueue A

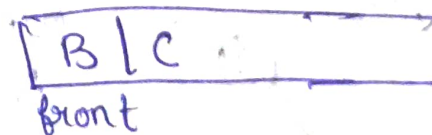


Step 2: Dequeue front

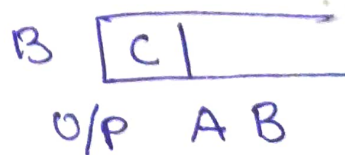
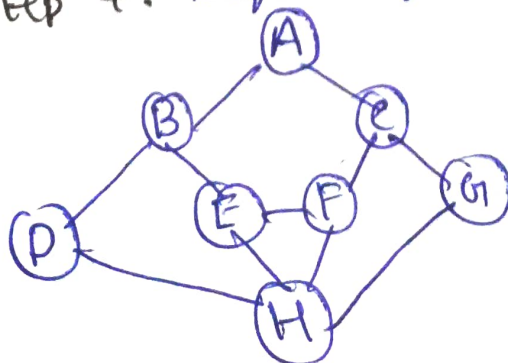


O/P: A

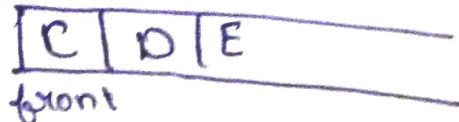
Step 3: Unvisited Neighbour of A → B, C  
Enqueue B, C



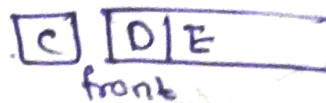
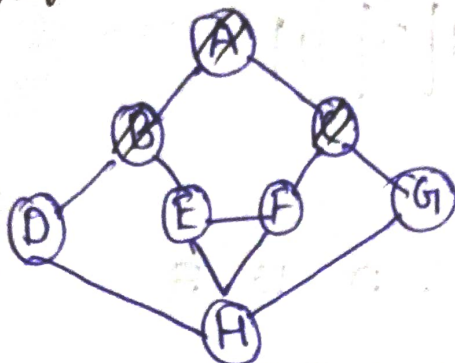
Step 4:- Dequeue front



Step 5: Unvisited Neighbours of B  $\rightarrow$  D, E  
 Enqueue D, E

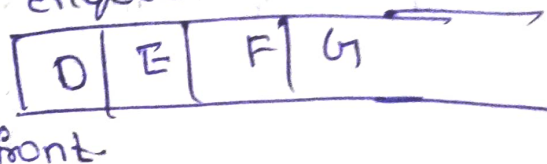


Step 6: Dequeue front

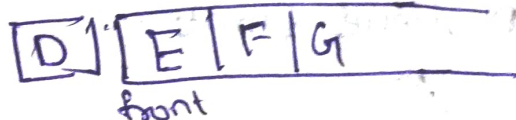
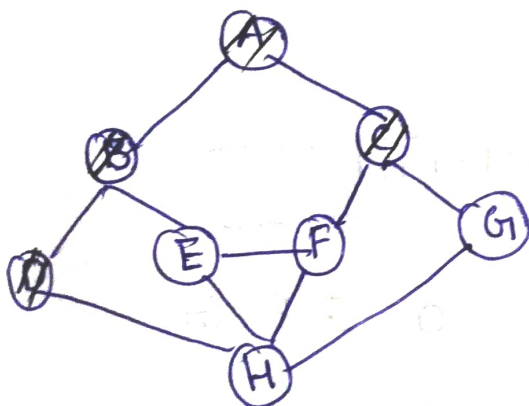


O/P: A B C

Step 7: Unvisited Neighbours of C  $\rightarrow$  F, G  
 Enqueue F, G

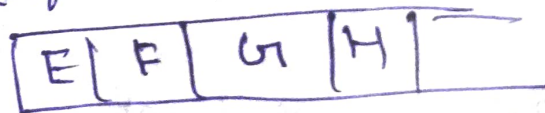


Step 8: Dequeue front

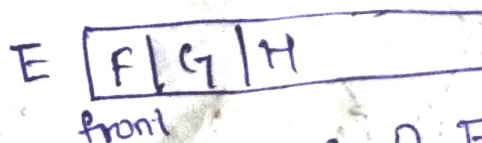
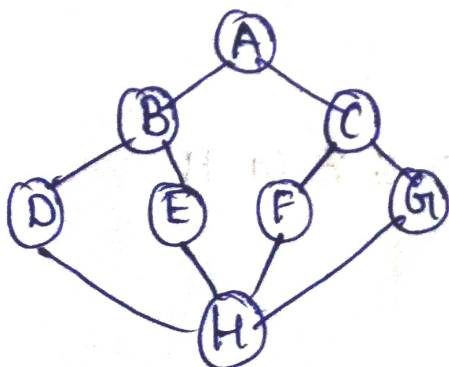


O/P: A B C D

Step 9: Unvisited Neighbours of D  $\rightarrow$  H  
 Enqueue H



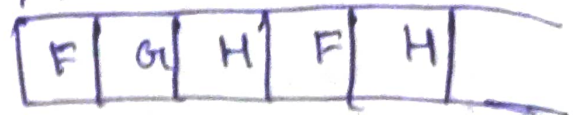
Step 10: Dequeue front



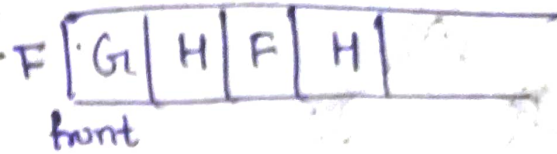
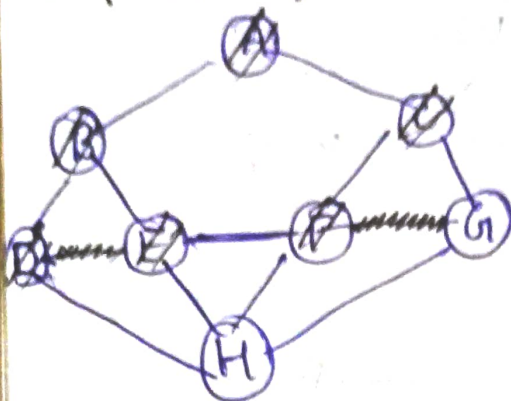
O/P: A B C D E

Step 11: Unvisited neighbour of  $E \rightarrow F, H$

Enqueue  $F, H$



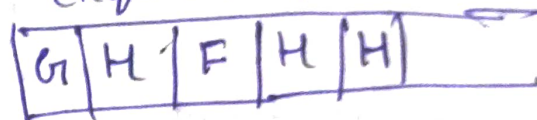
Step 12: Dequeue front



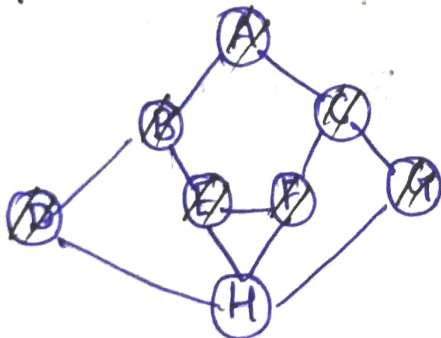
O/P!  
A B C D E F

Step 13: Unvisited neighbour of  $F \rightarrow H$

Enqueue  $H$

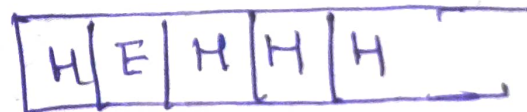


Step 14: Dequeue front

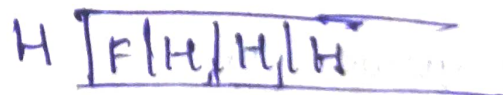
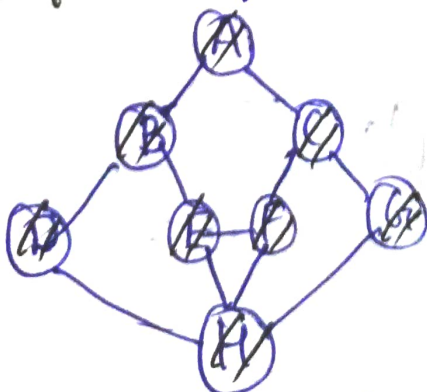


O/P  
A B C D E F G

Step 15: Unvisited neighbour of  $G \rightarrow H$

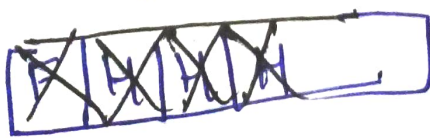


Step 16: Dequeue front

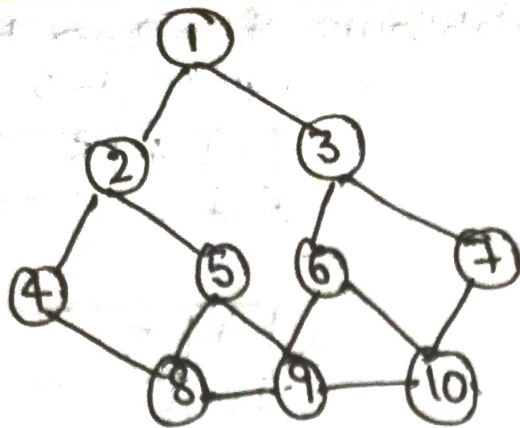


O/P!  
A B C D E F G H

Step 17: Dequeue front



2



Adjacency List:

1 [2, 3]

2 [1, 4, 5]

3 [1, 6, 7]

4 [2, 8]

5 [2, 8, 9]

6 [9, 10, 3]

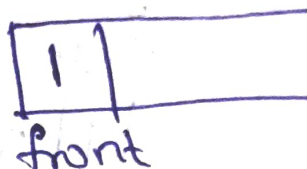
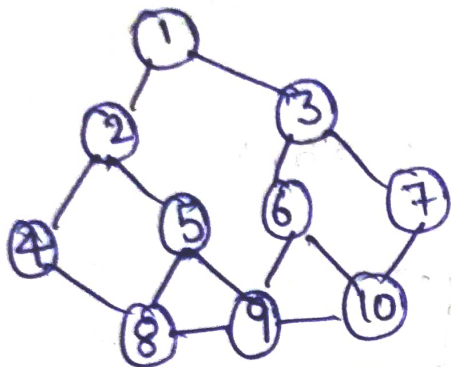
7 [3, 10]

8 [4, 5, 9]

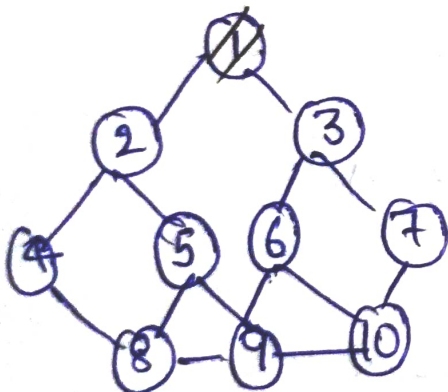
9 [5, 6, 8, 10]

10 [6, 7, 9]

Step 1:- Enqueue 1



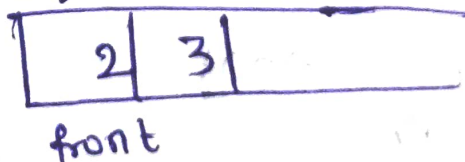
Step 2: dequeue front



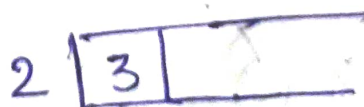
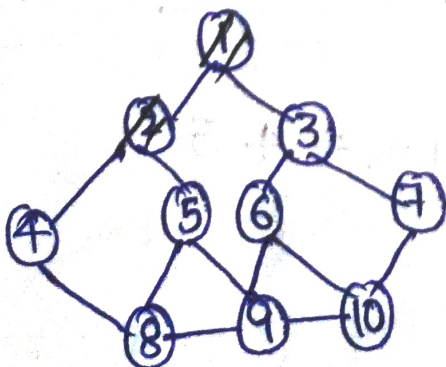
O/p: 1

Step 3 :- Unvisited Neighbours of 1  $\rightarrow$  2, 3

Enqueue 2, 3



Step 4: Dequeue front



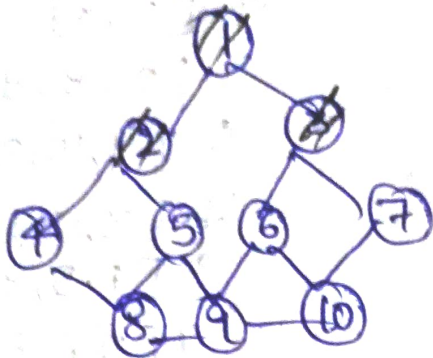
O/p: 1, 2

Step 5: Unvisited Neighbours of 2  $\rightarrow$  4, 5  
 Enqueue 4, 5

|   |   |   |
|---|---|---|
| 3 | 4 | 5 |
|---|---|---|

  
 front

Step 6: Dequeue front



|   |   |   |
|---|---|---|
| 3 | 4 | 5 |
|---|---|---|

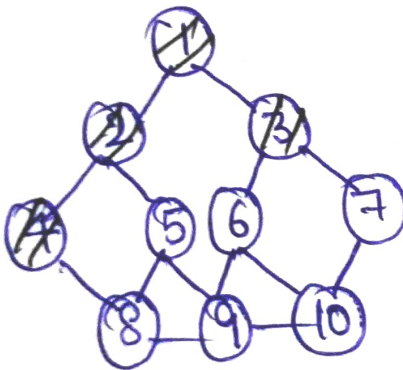
  
 front

O/p: 1 2 3

Step 7: Unvisited Neighbours of 3  $\rightarrow$  6, 7  
 Enqueue 6, 7

|   |   |   |   |
|---|---|---|---|
| 4 | 5 | 6 | 7 |
|---|---|---|---|

Step 8: Dequeue front



|   |   |   |   |
|---|---|---|---|
| 4 | 5 | 6 | 7 |
|---|---|---|---|

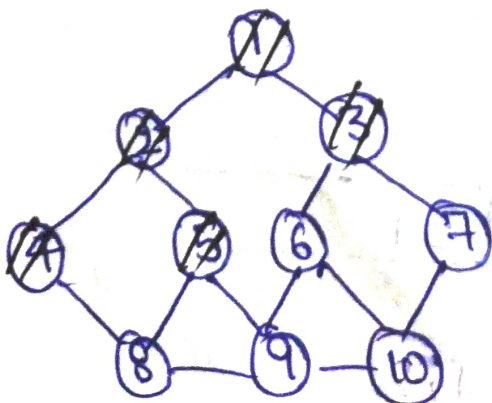
  
 front

O/p: 1 2 3 4

Step 9: Unvisited Neighbours of 4  $\rightarrow$  8  
 Enqueue 8

|   |   |   |   |
|---|---|---|---|
| 5 | 6 | 7 | 8 |
|---|---|---|---|

Step 10: Dequeue front



|   |   |   |   |
|---|---|---|---|
| 5 | 6 | 7 | 8 |
|---|---|---|---|

  
 front

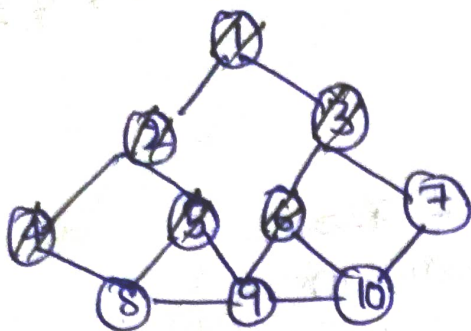
O/p: 1 2 3 4 5

Step 11: Unvisited neighbour of 5  $\rightarrow$  8, 9  
 Enqueue 8, 9

6 | 7 | 8 | 8 | 9  
 front

Step 12: Dequeue front

6 | 7 | 8 | 8 | 9

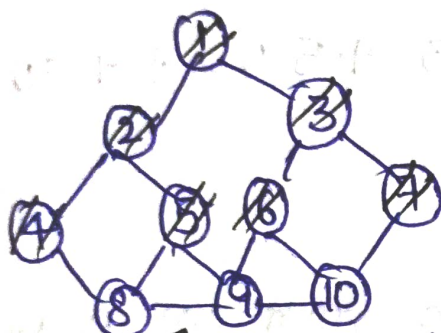


O/p: 1 2 3 4 5 6

Step 13: Unvisited neighbour of 6  $\rightarrow$  9, 10

Enqueue 9, 10

7 | 8 | 8 | 9 | 9 | 10



Step 14: Dequeue front

7 | 8 | 8 | 9 | 9 | 10

O/p: 1 2 3 4 5 6 7

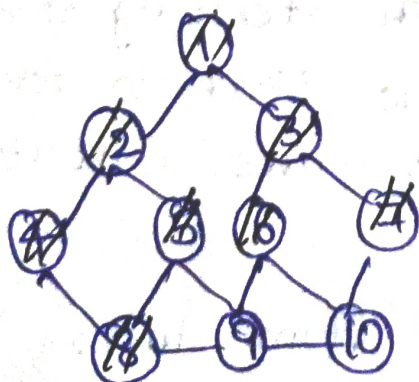
Step 15: Unvisited neighbours of 7  $\rightarrow$  10

Enqueue 10

8 | 8 | 9 | 9 | 10 | 10

Step 16: Dequeue front

8 | ~~8~~ | 9 | 9 | 10 | 10



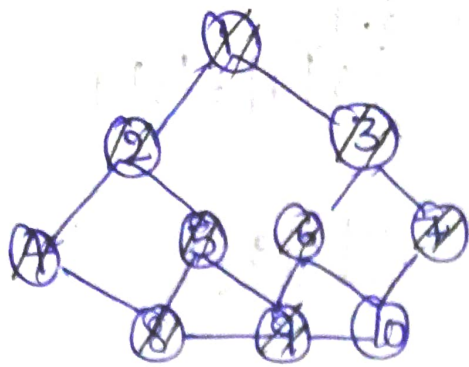
O/p: 1 2 3 4 5 6 7 8

Step 17: Unvisited neighbour of 8  $\rightarrow$  9

Enqueue 9

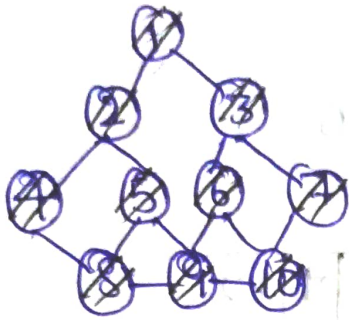
9 | 9 | 10 | 10 | 9

Step 18: Dequeue front



9 [ 9 | 10 | ]  
o/p: 1 2 3 4 5 6 7 8 9

Step 19: unvisited neighbours of 9  $\rightarrow$  10  
enqueue 10



10 [ 9 | 10 | ]

o/p: 1 2 3 4 5 6 7 8 9 10

### Dijkstra's Algorithm:

- $\rightarrow$  Finding the cheapest route from one node to every other node.
- $\rightarrow$  1956 invented by edges w. Dijkstra.
- $\rightarrow$  Greedy  $\rightarrow$  Always suspends the closed invalid node.
- $\rightarrow$   $\geq 0$  works when all edges weights are non negative.

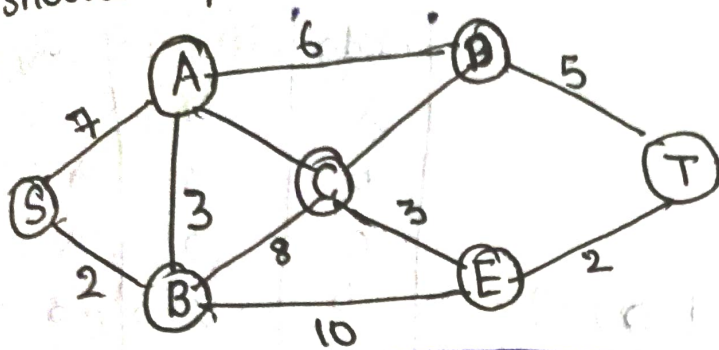
### Problem:-

- $\rightarrow$  Given a weighted graph and a starting node, find the shortest path from the start to every other node. This is the single source shortest path problem.
- $\rightarrow$  for every node shortest cost from start.
  1. Cost table
  2. Priority Queue: node with minimum cost is Popped.
  3. Relaxation: for an edge  $u \rightarrow v$

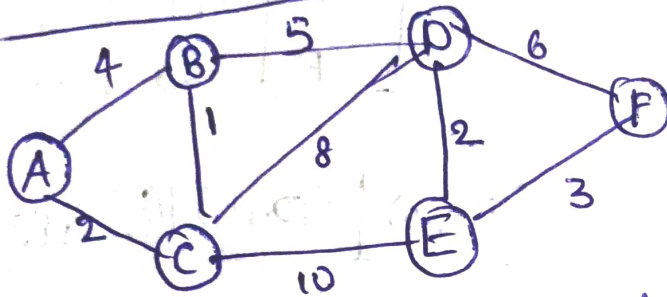
if  $\text{cost}[u] + w[u,v] < \text{cost}[v]$   
 update  $v$ 's cost and previous node.

4. Decrease key.

Q) shortest path from S



if  $d(u) + c(u,v) < d(v)$   
 then update.



$d(u) + c(u,v) < d(v) \Rightarrow \text{update}$

at A:

$$d(A) = 0$$

$A \rightarrow B$

$$0 + 4 < \infty \quad d(B) = 4$$

$A \rightarrow C$

$$0 + 2 < \infty$$

$$d(C) = 2$$

| Node | distance | Previous position |
|------|----------|-------------------|
| A    | 0        |                   |
| B    | 4        | A                 |
| C    | 2        | A                 |
| D    | $\infty$ |                   |
| E    | $\infty$ |                   |
| F    | $\infty$ |                   |

~~(B, 4)~~ ~~(C, 2)~~

(C, 2)

at C:  $d(C) = 2$

$C \rightarrow A$

$$2 + 2 < 0 \quad (\text{X})$$

$C \rightarrow B$

$$2 + 1 < 4 \quad 3 < 4$$

$$d(B) = 3$$

| Node | distance | Previous position |
|------|----------|-------------------|
| A    | 0        |                   |
| B    | 3        | C                 |
| C    | 2        | A                 |
| D    | 10       | C                 |
| E    | 12       | C                 |
| F    | $\infty$ |                   |

$$C \rightarrow D$$

$$2+8 < \infty \quad d(D)=10$$

$$C \rightarrow B$$

$$2+10 < \infty \quad d(E)=12$$

At B:  $d(B)=3$

$$B \rightarrow A:$$

$$3+4 < \infty \quad (\times)$$

$$B \rightarrow D$$

$$3+5 < 10 \quad 8 < 10 \quad d(D)=8$$

$$B \rightarrow C$$

$$3+1 < 2 \quad (\times)$$

At D:  $d(D)=8$

$$D \rightarrow B$$

$$8+5 < 3 \quad (\times)$$

$$D \rightarrow C:$$

$$8+8 < 2 \quad (\times)$$

$$D \rightarrow E:$$

$$8+2 < 12 \quad d(E)=10$$

$$D \rightarrow F$$

$$8+6 < \infty$$

$$14 < \infty \quad d(F)=14$$

At E:  $d(E)=10$

$$E \rightarrow C:$$

$$10+10 < 2 \quad (\times)$$

$$E \rightarrow D:$$

$$10+2 < 8 \quad (\times)$$

$$E \rightarrow F$$

$$10+3 < 14$$

$$d(F)=13$$

~~$(B, 3) (D, 10) (E, 12)$~~

(B, 3)

| Node | distance        | Prev node |
|------|-----------------|-----------|
| A    | 0               | -         |
| B    | 3               | A         |
| C    | 2               | A         |
| D    | <del>10</del> 8 | B         |
| E    | 12              | C         |
| F    | $\infty$        | -         |

~~$(B, 3) (D, 10) (E, 12) (D, 8)$~~

$d(8)$   
 $(D, 8)$

| Node | distance         | previous       |
|------|------------------|----------------|
| A    | 0                | -              |
| B    | 3                | C              |
| C    | 2                | A              |
| D    | 8                | B              |
| E    | <del>12</del> 10 | <del>A</del> D |
| F    | <del>14</del>    | D              |

~~$(E, 12) (D, 8) (E, 10) (F, 14)$~~

$(E, 10)$

| Node | Distance         | Previous       |
|------|------------------|----------------|
| A    | 0                | -              |
| B    | 3                | C              |
| C    | 2                | A              |
| D    | 8                | B              |
| E    | 10               | D              |
| F    | <del>14</del> 13 | <del>D</del> E |

(F, 13)

~~$(F, 14) (F, 13)$~~

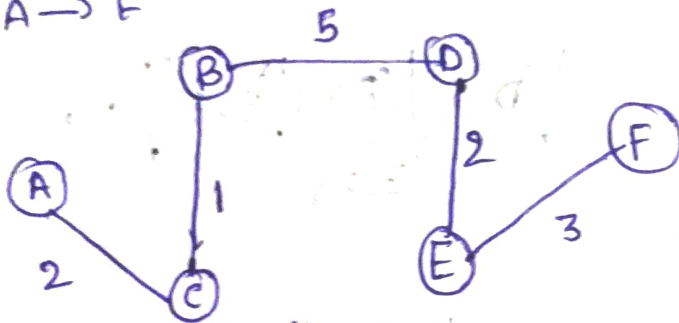
At F:  $d(F) = 13$

F → D:  
 $13 + 6 < 8$  (Y)

F → E:  
 $13 + 3 < 10$  (Y)

shortest distance:

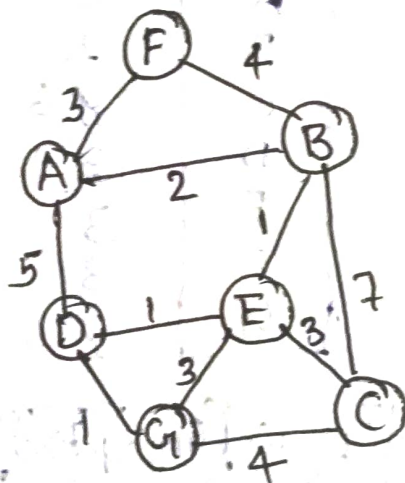
A → F



$$2 + 5 + 1 + 2 + 3 = 13$$

| Node | distance | previous |
|------|----------|----------|
| A    | 0        |          |
| B    | 3        | C        |
| C    | 2        | A        |
| D    | 8        | B        |
| E    | 10       | D        |
| F    | 13       | E        |

3)



$$d(u) + c(u, v) < d(v) \text{ update:}$$

at A:  $d(A) = 0$

A → B:  
 $0 + 2 < \infty$   $d(B) = 2$

A → F:  
 $0 + 3 < \infty$   $d(F) = 3$

A → D:  
 $0 + 5 < \infty$   
 $d(D) = 5$

| Node | distance       | Prev |
|------|----------------|------|
| A    | 0              |      |
| B    | <del>∞</del> 2 | A    |
| C    | ∞              |      |
| D    | <del>∞</del> 5 | A    |
| E    | ∞              |      |
| F    | <del>∞</del> 3 | A    |
| G    | ∞              |      |

(B, 2) (D, 5) (F, 3)

(B, 2)

at B:  $d(B)=2$

B → D:

$$2+4 < 3 \quad (\times)$$

B → A:

$$2+2 < 0 \quad (\times)$$

B → E:

$$2+1 < \infty$$

$$d(E)=3$$

B → C:

$$2+7 < \infty$$

$$9 < \infty \quad d(C)=9$$

at E:  $d(E)=3$

E → B:

$$3+1 < 2 \quad (\times)$$

E → D:

$$3+1 < 5$$

$$d(D)=4$$

E → C:

$$3+3 < 9$$

$$6 < 9$$

$$d(C)=6$$

E → G:

$$3+3 < \infty \quad d(G)=6$$

at F:  $d(F)=3$

F → A:

$$3+3 < 0 \quad (\times)$$

F → B:

$$3+4 < 2 \quad (\times)$$

| Node | Distance     | Prev |
|------|--------------|------|
| A    | 0            |      |
| B    | 2            | A    |
| C    | <del>9</del> | B    |
| D    | 5            | A    |
| E    | <del>3</del> | B    |
| F    | 3            | A    |
| G    | $\infty$     |      |

|        |        |                   |        |
|--------|--------|-------------------|--------|
| (D, 5) | (F, 3) | <del>(E, 3)</del> | (C, 9) |
|        | (E, 3) |                   |        |

| Node | Distance                         | Prev           |
|------|----------------------------------|----------------|
| A    | 0                                |                |
| B    | 2                                | A              |
| C    | <del>9</del> 6                   | <del>B</del> E |
| D    | <del>5</del> 4                   | <del>A</del> E |
| E    | 3                                | B              |
| F    | 3                                | A              |
| G    | <del><math>\infty</math></del> 6 | E              |

|        |        |                   |        |
|--------|--------|-------------------|--------|
| (D, 4) | (G, 6) | <del>(F, 3)</del> | (C, 6) |
|        | (F, 3) |                   |        |

| Node | Distance | Prev |
|------|----------|------|
| A    | 0        |      |
| B    | 2        | A    |
| C    | 6        | E    |
| D    | 4        | E    |
| E    | 3        | B    |
| F    | 3        | A    |
| G    | 6        | E    |

|        |                   |        |        |
|--------|-------------------|--------|--------|
| (D, 4) | <del>(D, 4)</del> | (G, 6) | (C, 6) |
|--------|-------------------|--------|--------|

At D:  $d(D) = 4$

D → A:  
 $4 + 5 < 0$  (X)

D → E:  
 $4 + 1 < 3$  (Y)

D → G:  
 $4 + 1 < 6$   
 $d(G) = 5$

At G:  $d(G) = 5$

G → D:  
 $5 + 1 < 4$  (Y)

G → E:  
 $5 + 3 < 3$  (X)

G → C:  
 $5 + 4 < 6$  (X)

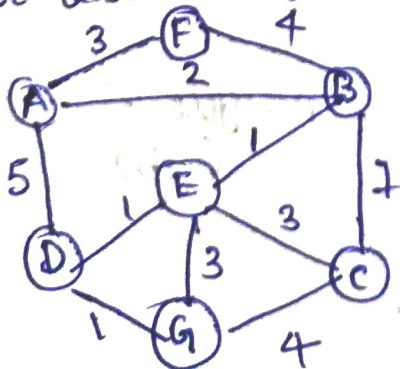
At C:  $d(C) = 6$

C → B:  
 $6 + 7 < 2$  (X)

C → E:  
 $6 + 3 < 3$  (X)

C → G:  
 $6 + 5 < 5$  (X)

Shortest distance from A → C



| Node | distance | Prev |
|------|----------|------|
| A    | 0        |      |
| B    | 2        | A    |
| C    | 6        | E    |
| D    | 4        | E    |
| E    | 3        | B    |
| F    | 3        | A    |
| G    | 5        | D    |

~~(G, 5)~~ (C, 6)

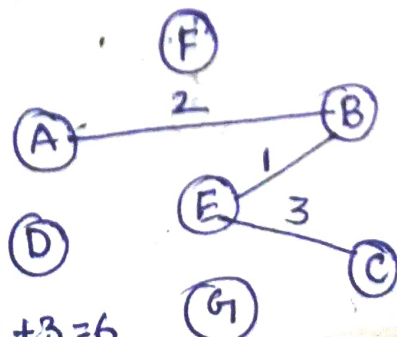
(G, 5)

| Node | Distance | Prev |
|------|----------|------|
| A    | 0        |      |
| B    | 2        | A    |
| C    | 6        | E    |
| D    | 4        | E    |
| E    | 3        | B    |
| F    | 3        | A    |
| G    | 5        | D    |

~~(C, 6)~~

(C, 6)

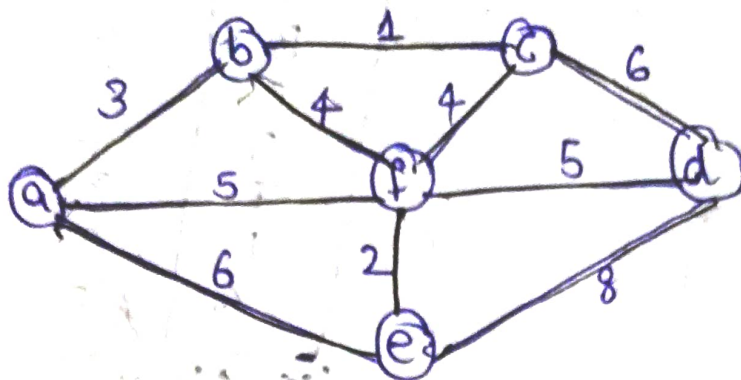
| Node | distance | Prev |
|------|----------|------|
| A    | 0        |      |
| B    | 2        | A    |
| C    | 6        | E    |
| D    | 4        | E    |
| E    | 3        | B    |
| F    | 3        | A    |
| G    | 5        | D    |



$2 + 1 + 3 = 6$

# Kruskal Algorithm:

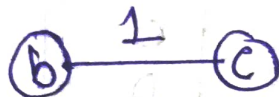
1.



| bc | ef | ab | bf | cf | af | df | ae | cd | de |
|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 4  | 5  | 5  | 6  | 6  | 8  |
| ✓  | ✓  | ✓  | ✓  | X  | X  | ✓  | X  | X  | X  |

$|V|=6$ ,  $|E|=10$ , 5 edges

(i)



no cycle ✓

(bc)

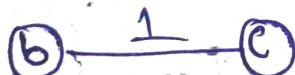
(a)

(f)

(d)

(e)

(ii)



no cycle ✓  
(ef)

(a)

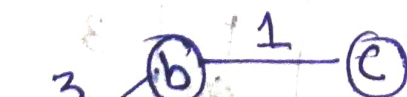
(f)

(d)

2

(e)

(iii)



no cycle ✓  
(ab)

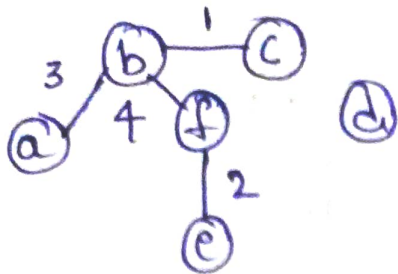
(a)

(f)

(d)

(e)

(iv)

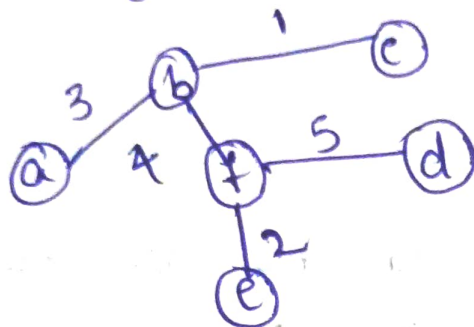


no cycle ✓  
(bf)

(v) cf forms a cycle so skip

(vi) af forms a cycle so skip

(vii) df no cycle



(viii) ae forms a cycle so skip.

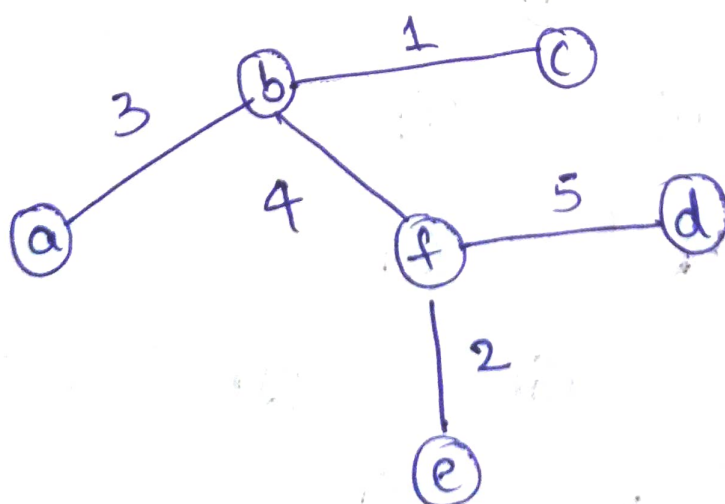
(ix) ed forms a cycle so skip

(x) de forms a cycle so skip

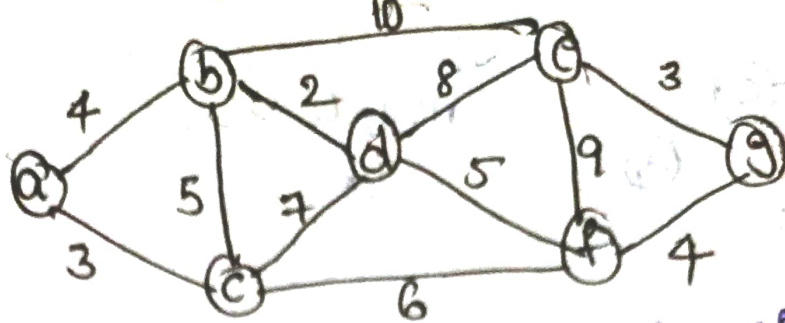
minimum spanning tree weight

$$= 1 + 2 + 3 + 4 + 5$$

$$= 15$$



2)



|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| ab | ac | bc | bd | be | cd | cf | de | df |
| 4  | 3  | 5  | 2  | 10 | 7  | 6  | 8  | 5  |
| cf | eg | fg |    |    |    |    |    |    |
| 9  | 3  | 4  |    |    |    |    |    |    |

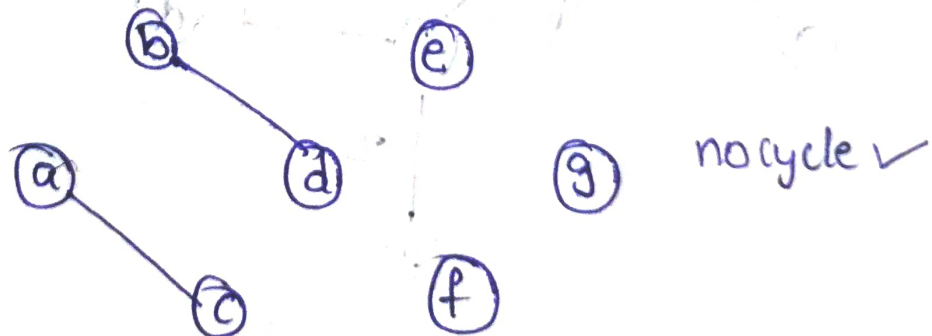
Sorted :-

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| bd | ac | eg | ab | fg | bc | df | cf | cd |
| 2  | 3  | 3  | 4  | 4  | 5  | 5  | 6  | 1  |
| ✓  | ✓  | ✓  | ✓  | ✓  | x  | ✓  | x  | x  |
| de | ef | be |    |    |    |    |    |    |
| 8  | 9  | 10 |    |    |    |    |    |    |
| x  | x  | x  |    |    |    |    |    |    |

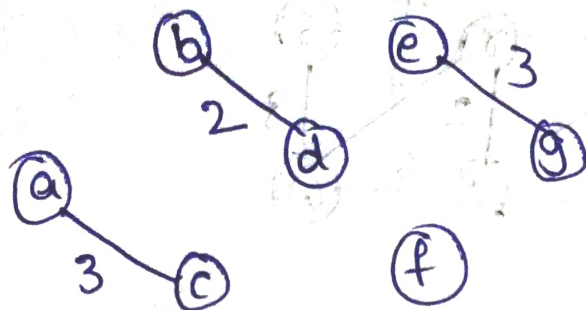
(i)



(ii)

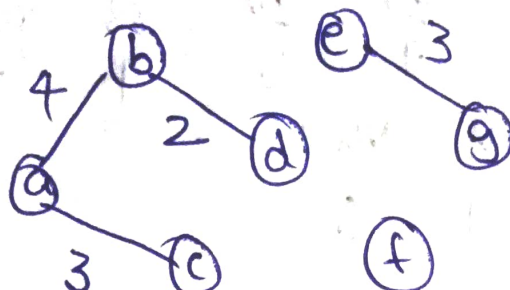


(iii)



no cycle ✓

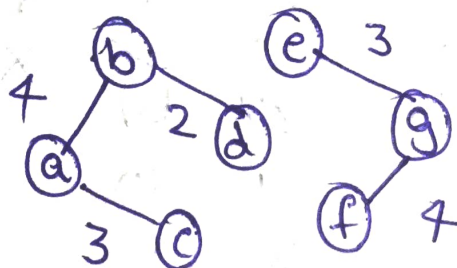
(iv)



no cycle

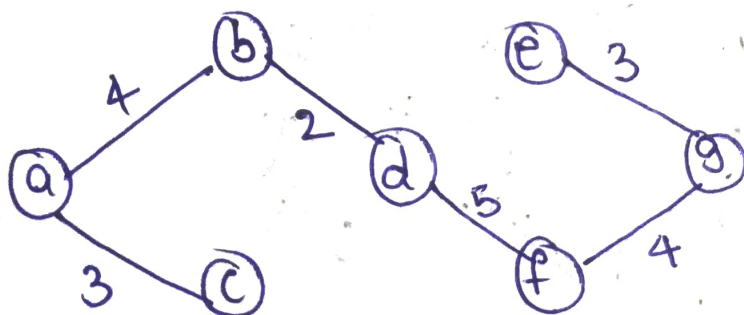
(v) bc forms a cycle so skip

(vi)



no cycle

(vii)



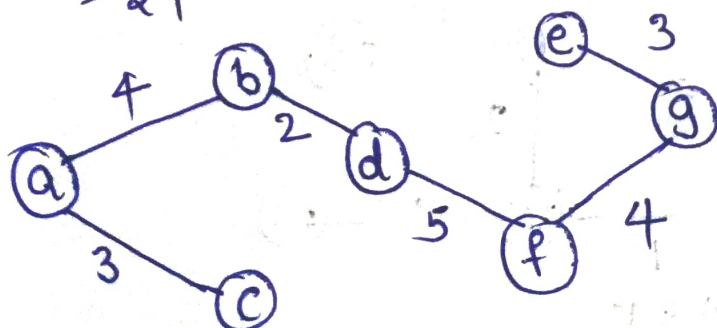
no cycle

(viii) cf, cd, de, ef, be form a cycle so skip.

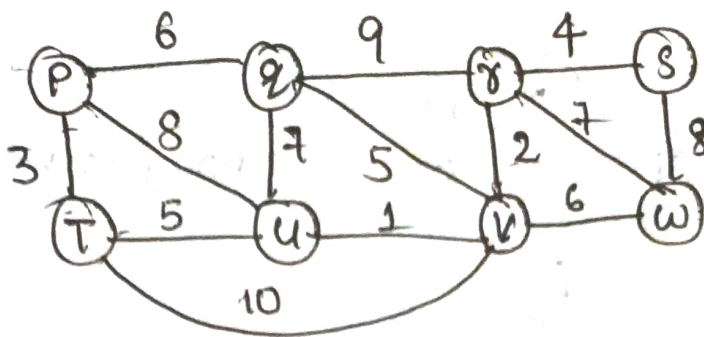
minimum spanning tree's weight

$$= 2 + 3 + 3 + 4 + 4 + 5$$

$$= 21$$



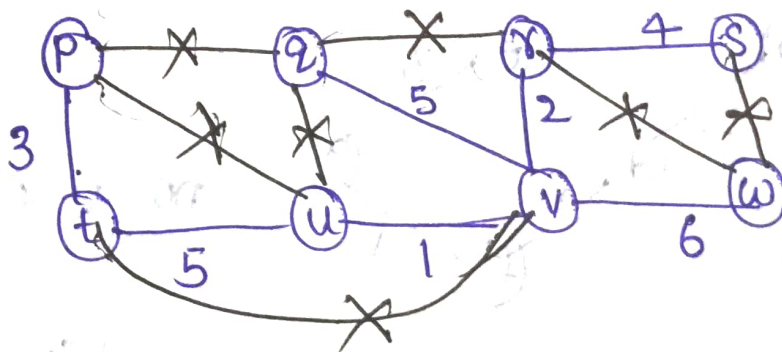
3



|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| PT | PQ | PR | QR | QU | QV | RS | RW | RV |
| 3  | 6  | 9  | 7  | 5  | 4  | 7  | 2  |    |
| sw | tu | tv | uv | vw |    |    |    |    |
| 8  | 5  | 10 | 1  | 6  |    |    |    |    |

Sosited:-

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| uv | rv | pf | rs | qv | tu | pq | vw |
| 1  | 2  | 3  | 4  | 5  | 5  | 6  | 6  |
| ✓  | ✓  | ✓  | ✓  | ✓  | ✓  | x  | ✓  |
| qu | rw | pu | sw | qr | tv |    |    |
| 7  | 7  | 8  | 8  | 9  | 10 |    |    |
| x  | x  | x  | x  | x  | x  |    |    |



minimum Spanning tree's weight

$$= 1 + 2 + 3 + 4 + 5 + 5 + 6$$

$$= 26$$

